

데이터 수집: Open API

#Selenium #OpenAPI #API

목차

- 01 웹 크롤링 확장: Selenium
- 02 OpenAPI란
- 03 API의 구조
- 04 OpenAPI 활용



웹 크롤링 확장: Selenium

기존 웹 크롤링의 한계!

BeautifulSoup는 이미 완성된 HTML만 분석한다.

하지만 많은 웹 사이트는 JavaScript를 통해 페이지가 동적으로 생성된다!

ex)
더보기 버튼 클릭 시 데이터 로드
스크롤할 때마다 추가 콘텐츠가 나타나는 형태
로그인이 필요한 경우

화면 상에는 보이지만 HTML에는 보이지 않음!

Selenium

동적으로 웹페이지를 제어하기 위한 라이브러리



브라우저 제어(크롬, 엣지, 파이어폭스, 사파리)

클릭, 입력, 스크롤, 드래그와 같은
실제 사용자 행동과 동일한 동작 수행

JS로 동적으로 생성되는 페이지 처리 가능

라이브러리 준비단계

라이브러리 설치하기

```
pip install selenium
```

라이브러리 불러오기 및 웹 드라이버 불러오기

```
from selenium import webdriver  
  
driver = webdriver.Chrome()
```

**예전에는 별도로 웹드라이버를 설치해야 했지만,
현재는 브라우저에 맞춰 자동으로 설치 가능*

Driver?

Selenium과 브라우저 사이의 통신을 담당하는 중간 제어 프로그램
실제 브라우저를 조작하게 하는 역할

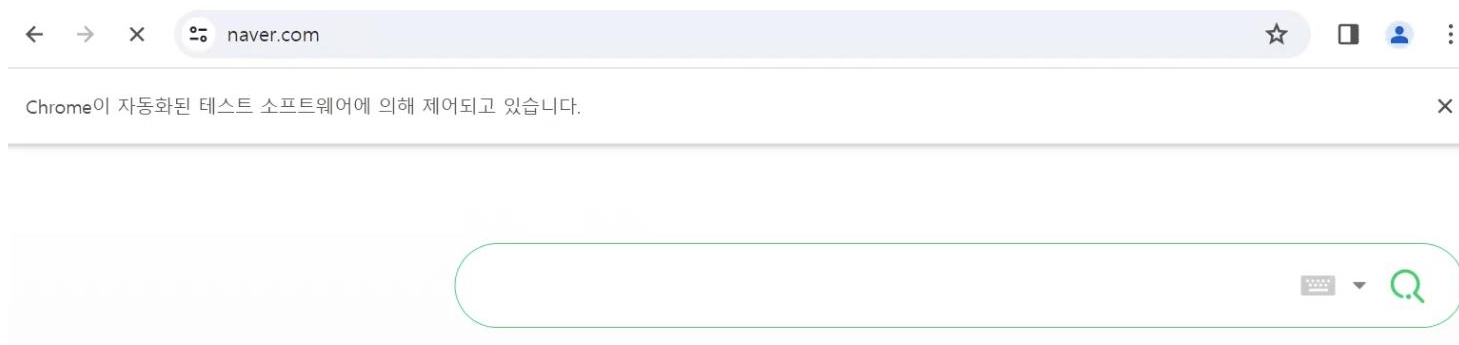
직접 사용해보자!

특정 URL에 대해 웹 드라이버를 통해 접속

```
from selenium import webdriver

driver = webdriver.Chrome()

url = "https://naver.com"
driver.get(url)
```



실행하면 실제로 자동으로 브라우저가 열림

직접 사용해보자!

옵션: 화면이 꺼지지않고 계속 유지하기

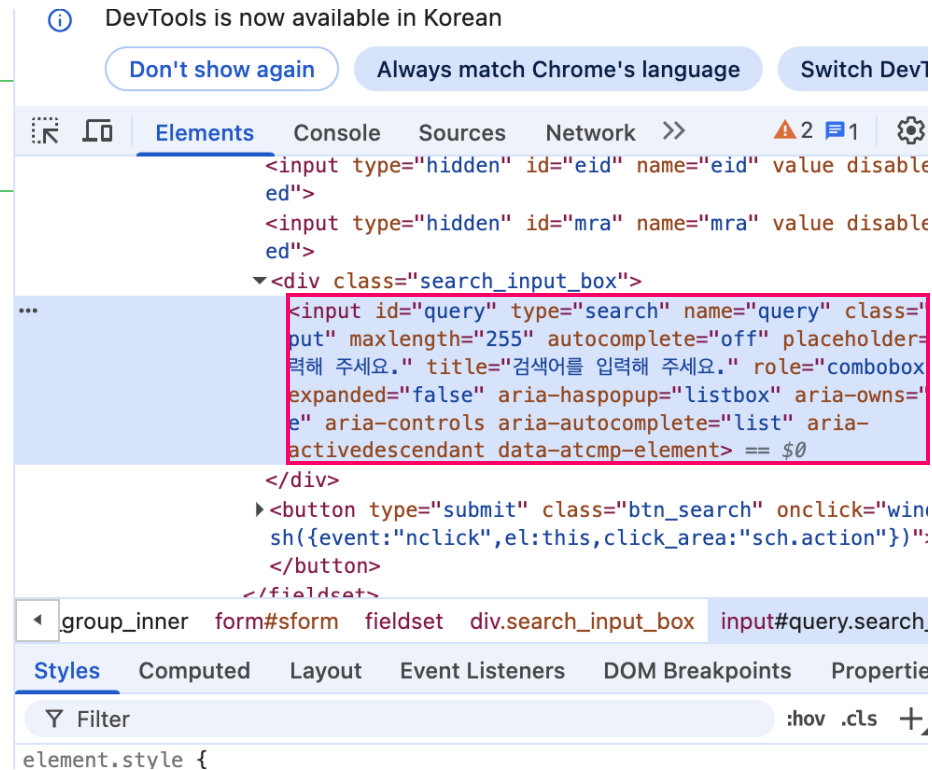
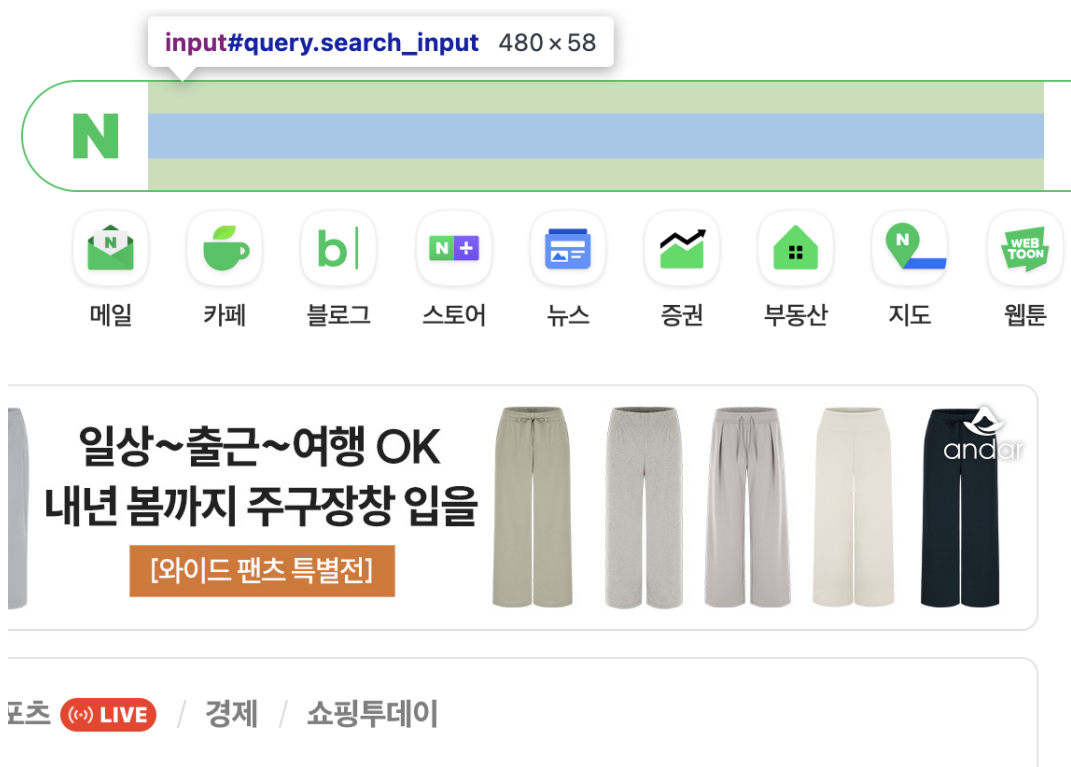
```
from selenium import webdriver
from selenium.webdriver.chrome.options import Options

options = Options()
options.add_experimental_option("detach", True)

driver = webdriver.Chrome(options=options)
```


직접 사용해보자: 입력과 버튼 클릭

검색 창의 태그를 개발자 모드에서 찾기



직접 사용해보자: 입력과 버튼 클릭

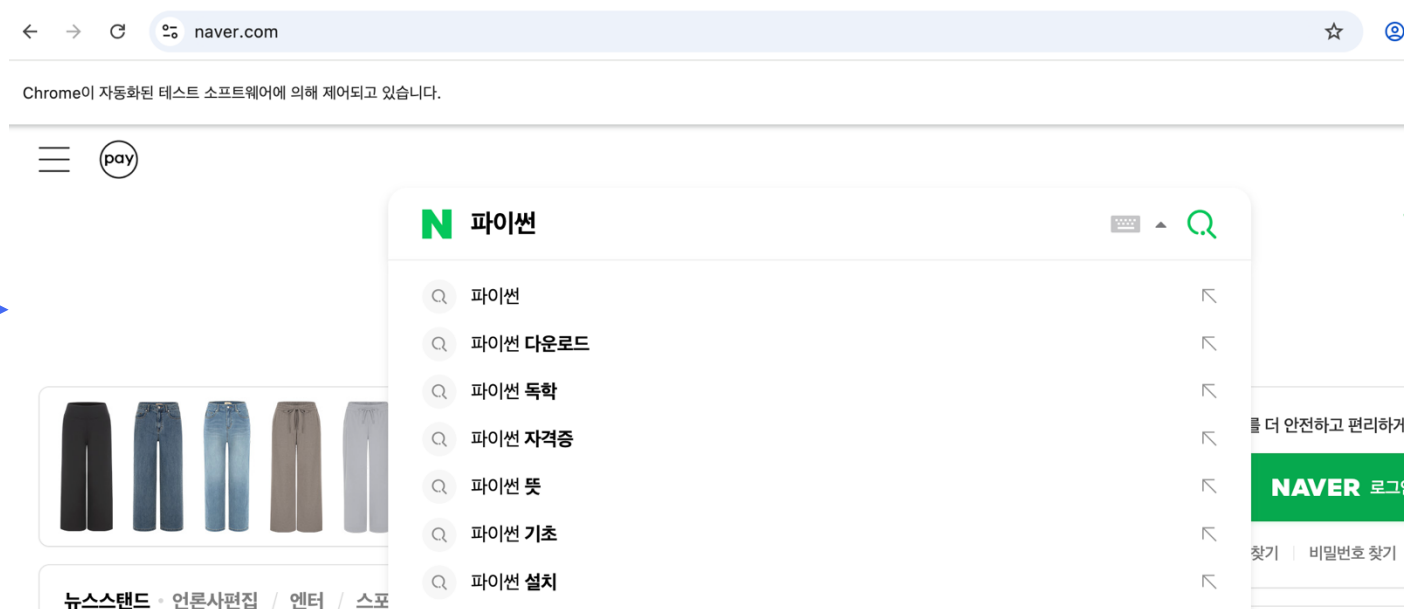
해당 요소에 키보드 입력하기

```
query = driver.find_element(By.ID, "query")  
#query = driver.find_element(By.CSS_SELECTOR, "#query")  
query.sends_key("파이썬")
```

find_element() 함수로 id 속성이 "query"인 요소 객체 반환

sends_key() 함수를 바탕으로 특정 키 입력

자동으로 입력 창에 "파이썬"이 입력됨



직접 사용해보자: 입력과 버튼 클릭

time 모듈을 활용하여 브라우저 로딩 등 작업에 대한 대기시간 추가

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.options import Options
import time

options = Options()
options.add_experimental_option("detach", True)

driver = webdriver.Chrome(options=options)

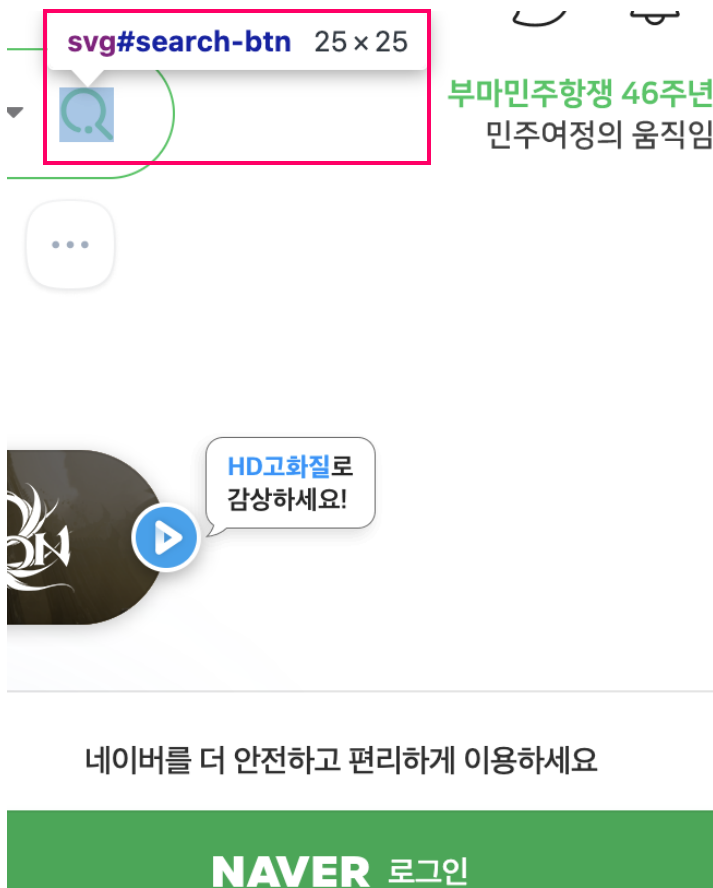
url = "https://naver.com"
driver.get(url)
time.sleep(2)

query = driver.find_element(By.ID, "query")
#query = driver.find_element(By.CSS_SELECTOR, "#query")
query.send_key("파이썬")
```

2초 대기 후, 다음 코드 실행

직접 사용해보자: 입력과 버튼 클릭

버튼 클릭 자동화 해보기: 버튼에 해당하는 태그 찾기



svg#search-btn 25 x 25

부마민주항쟁 46주년
민주여정의 움직임

HD고화질로
감상하세요!

네이버를 더 안전하고 편리하게 이용하세요

NAVER 로그인

Don't show again Always match Chrome's language Switch DevTools to Korean

Elements Console Sources Network >> 2 1

```
<input type="hidden" id="pkid" name="pkid" value disabled="disabled">
<input type="hidden" id="eid" name="eid" value disabled="disabled">
<input type="hidden" id="mra" name="mra" value disabled="disabled">
▶ <div class="search_input_box"> ... </div>
▼ <button type="submit" class="btn_search" onclick="window.ntm.push({event:"nclick",el:this,click_area:"sch.action"})">
  ▼ <span class="ico btn_search svg">
    ▼ <svg id="search-btn" viewBox="0 0 50 50" xmlns="http://www.w3.org/2000/svg" style="fill: #03c75a">
      <path d="M22.13,5c11.378 0 20.632 9.256 20.632 20.63 0 4.699-1.566 9.155-4.439 12.782l10.164 10.165a2.41 2.41 0 0 1-1.706 4.115 2.412 2.412 0 0 1-1.706-.705L33.31 35.719a2.41 2.41 0 0 1 0-3.41 15.71 15.71 0 0 0 4.628-11.178c0-8.718-7.09-15.808-15.807-15.808-8.718 0-15.808 7.09-15.808 15.808 0 7.15 4.817 13.43 11.714 15.273a2.41 2.41 0 0 1 1.705 2.954 2.41 2.41 0 0 1-2.95 1.705C7.788 38.658 1.5 30.46 1.5 21.131 1.5 9.756 10.756 0 22.13,5zm4.716 34.746a3.483 3.483 0 1 1 0 6.966 3.483 3.483 0 0 1 0-6.966z">
    </path> == $0
```

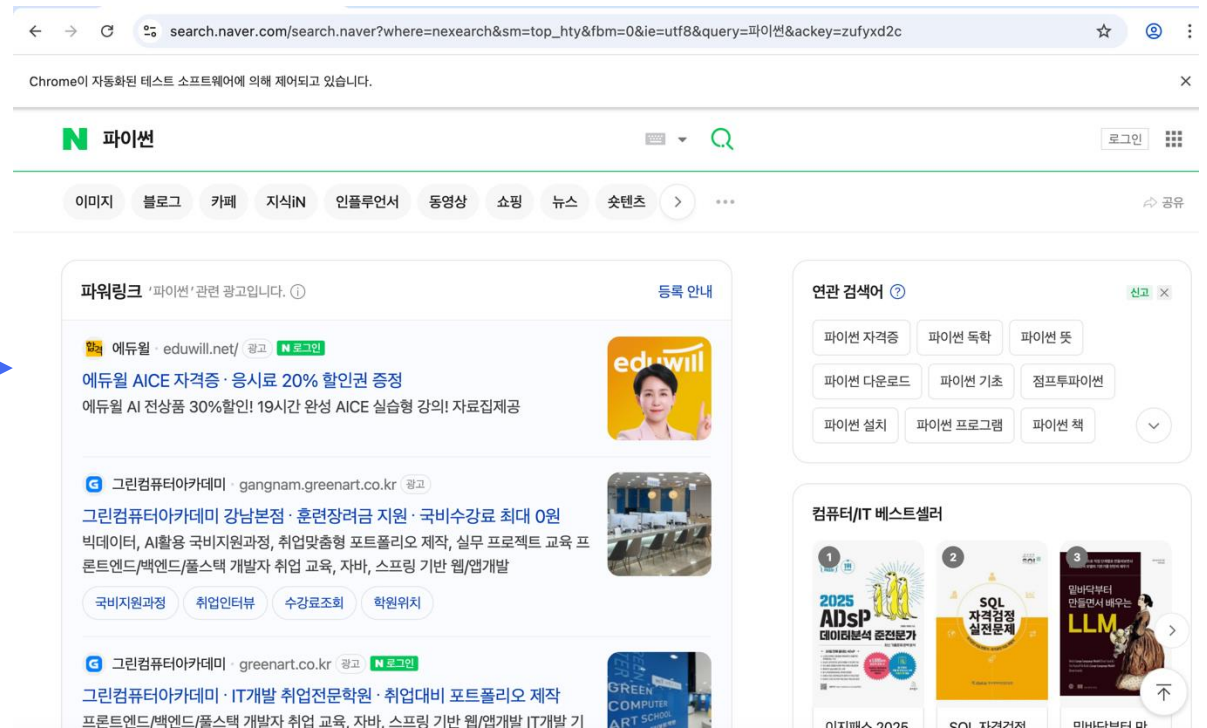
직접 사용해보자: 입력과 버튼 클릭

버튼 클릭 자동화 해보기: 해당 요소를 가져와 클릭 수행

```
search_btn = driver.find_element(By.CSS_SELECTOR, "#search-btn")
search_btn.click()
```

id속성이 "search-btn"인 요소 객체 반환 후,
해당 객체에 대해 `click()` 메서드로 클릭 수행

자동으로 입력 창에 "파이썬" 입력 후,
검색 버튼 클릭



직접 사용해보자: 입력과 버튼 클릭

이번엔 버튼 대신 엔터키를 입력하기

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.common.keys import Keys
import time

options = Options()
options.add_experimental_option("detach", True)

driver = webdriver.Chrome(options=options)

url = "https://naver.com"
driver.get(url)
time.sleep(2)

query = driver.find_element(By.ID, "query")
query.send_key("파이썬")
time.sleep(2)
query.send_key(Keys.ENTER)
```

Selenium의 요소 검색

find_element() 함수를 활용하여 요소 찾기

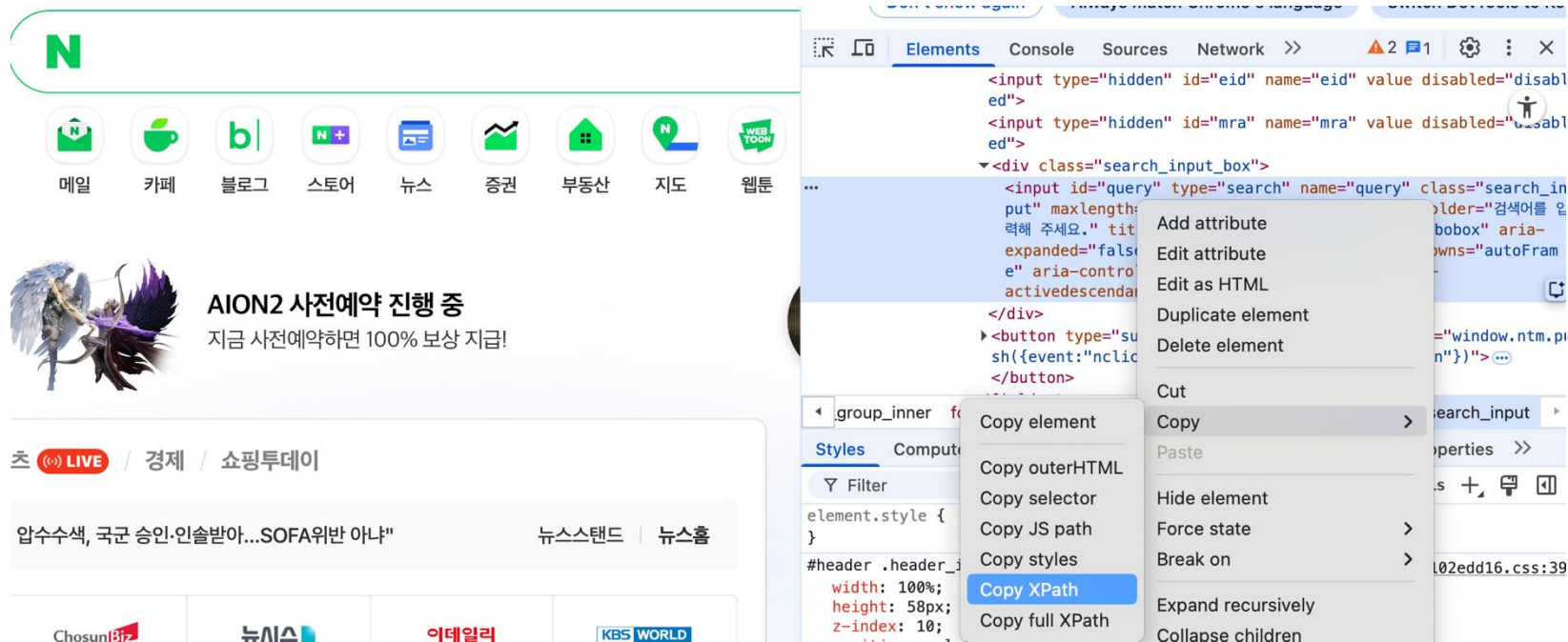
find_element(**로케이터**, **값**)

로케이터	설명
By.TAG_NAME	<p>, <div> 등 태그 이름으로 검색
By.ID	태그의 id 속성으로 검색
By.CLASS_NAME	태그의 class 속성으로 검색
By.CSS_SELECTOR	css 선택자로 검색
By.XPATH	태그의 경로로 검색 (시작부터 목표한 요소까지 태그를 타고 내려온 경로)
By.NAME	태그의 name 값으로 검색

Selenium의 요소 검색

XPATH? 특정 요소를 찾기 위한 경로 표현식

```
query = driver.find_element(By.XPATH, '//*[@id="query"]')
```



태그에 class나 id가 없어서 찾기 어려운 경우 유용

[요소 우클릭 > Copy Xpath]

조작 수행 메서드

웹 드라이버를 통해 원하는 요소 객체를 얻고, 해당 객체에 대해 동작 수행

- 클릭(좌클릭): **click()**
- 키 입력(텍스트, 엔터 등) : **send_keys()**
- 입력 내용 삭제: **clear()**
- <form> 요소 전송: **submit()**

*그 외 드라이버의 동작 함수

- **driver.refresh()**: 새로고침
- **driver.back()**: 뒤로 가기
- **driver.forward()**: 앞으로 가기
- **driver.save_screenshot("page.png")**: 현재 화면 캡처

키 종류	기능
Keys.ENTER, Keys.RETURN	엔터
Keys.SPACE	스페이스
Keys.ARROW_UP Keys.ARROW_DOWN Keys.ARROW_LEFT Keys.ARROW_RIGHT	방향키 (상하좌우)
Keys.BACK_SPACE Keys.DELETE	지우기(백스페이스)
	지우기(딜리트)
Keys.CONTROL	기능 키
Keys.ALT	
Keys.SHIFT	
Keys.PAGE_UP, Keys.PAGE_DOWN	스크롤 업, 다운

명시적 대기: WebDriverWait

`time.sleep()`은 파이썬 프로세스를 일시 정지하는 것(= 무조건 멈춰라)

WebDriverWait는 최대 대기시간 내에서, 조건이 만족되면 즉시 다음 단계로 진행
페이지 로딩 속도가 느리거나, JS 동적 로딩 시 유용

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.common.by import By

wait = WebDriverWait(driver, 10) # 최대 10초 동안 기다림

element = wait.until(
    EC.presence_of_element_located((By.CSS_SELECTOR, ".my-element"))
)
```

- wait 객체 생성
- wait 객체의 `until()` 메서드
 - 인자로 넣은 조건이 10초 안에 충족되면 즉시 다음 코드로 넘어가고
 - 10초 동안 조건이 만족되지 않으면

TimeoutException 발생

명시적 대기: WebDriverWait

주요 **Expected Conditions(EC)**: 여러 가지 상황에 대한 대기 조건

조건	설명
presence_of_element_located	(보이지 않아도)HTML에 요소가 존재하면 True
visibility_of_element_located	요소가 존재하고 보이면 True
element_to_be_clickable	클릭 가능한 상태면 True
text_to_be_present_in_element	지정한 텍스트가 요소 내 포함되면 True

```
EC.조건((로케이터, 값))
```

명시적 대기: WebDriverWait

Example

```
wait = WebDriverWait(driver, 10) # 최대 10초 대기

login_button = wait.until(
    EC.element_to_be_clickable((By.CSS_SELECTOR, "button.login"))
)
login_button.click()
```

“**login_button**”이 로딩 완료되어
클릭 가능한 상태일 때 True가 되어 실행

```
wait.until(
    EC.text_to_be_present_in_element((By.TAG_NAME, "h1"), "로그인 완료")
)
print("로그인 성공 페이지 확인됨!")
```

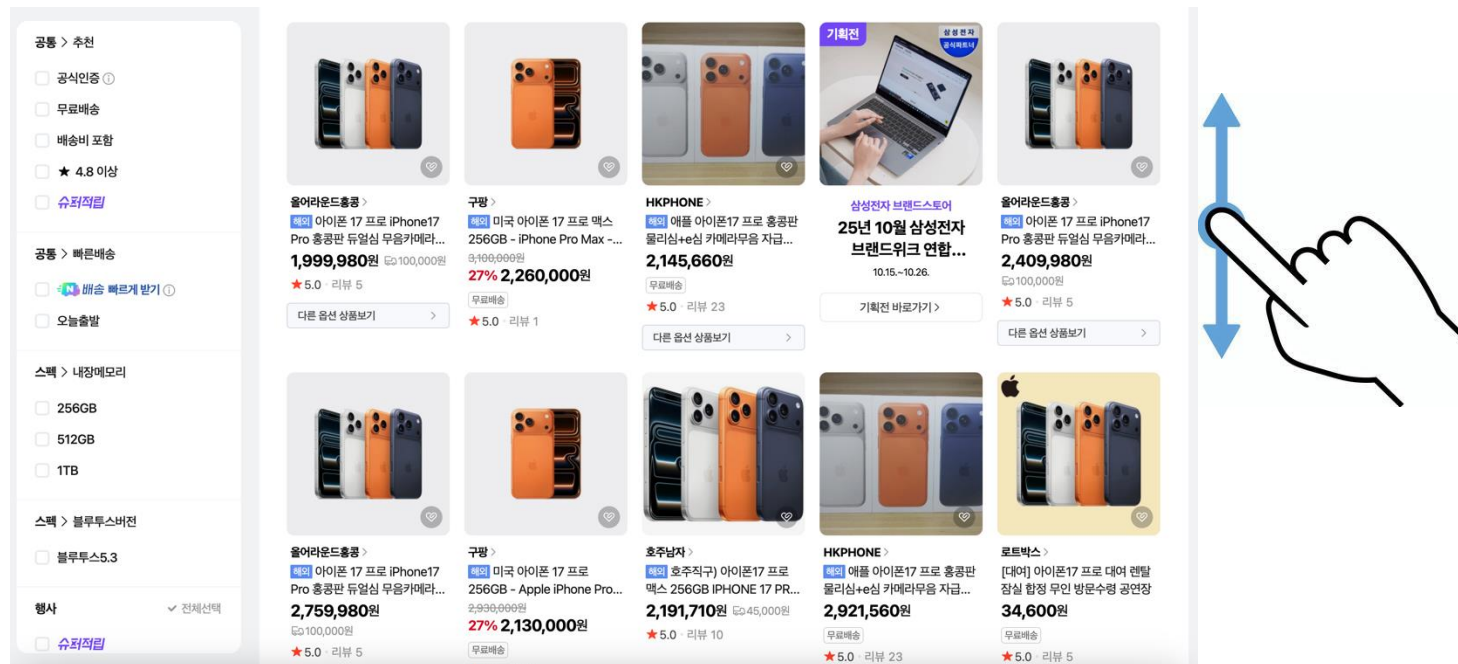
페이지의 **<h1>**에 “**로그인 완료**”
라는 문구가 나타나면 True

추가 활용: 무한 스크롤

지속적인 스크롤을 통해 아이템이 로딩되어 갱신하는 구조

```
driver.execute_script(SCRIPT)
```

웹 브라우저에 JavaScript를 동적으로 실행



추가 활용: 무한 스크롤

스크립트 실행

```
# (전제) 이미 driver.get(...)으로 페이지를 열고,  
# 첫 화면의 기본 요소가 보일 때까지 대기했다고 가정  
  
last_height = driver.execute_script("return document.body.scrollHeight")  
  
while True:  
    driver.execute_script("window.scrollTo(0, document.body.scrollHeight);")  
    time.sleep(2)  
    new_height = driver.execute_script("return document.body.scrollHeight")  
    if new_height == last_height:  
        break  
    last_height = new_height
```

1. 문서의 스크롤 가능한 전체 높이(px) 반환
2. 무한 루프를 돌며, 문서 맨 아래로 이동
3. 스크롤 후, 새로 계산된 문서 전체 높이 반환
4. 높이 비교 후, 같으면 무한 스크롤 종료(=높이 변화 X)

여기서는 모든 아이템을 가져올 때까지 스크롤하는 것이지만, 그 외의 다른 조건으로 응용 가능

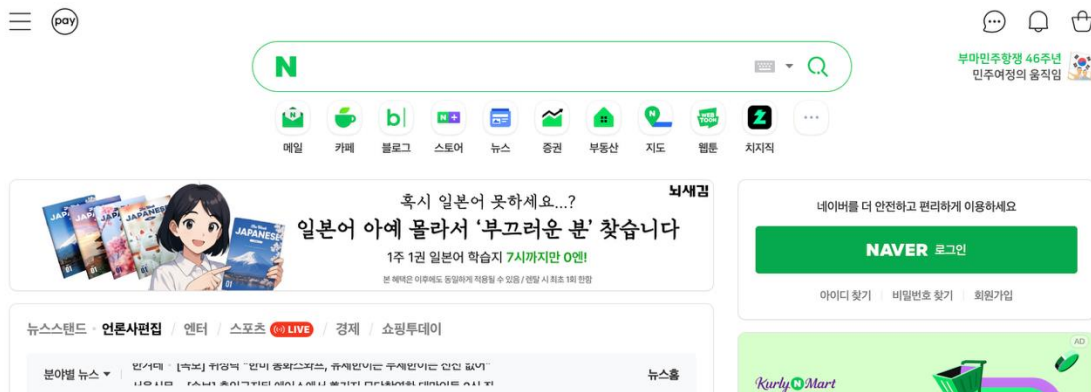
동적 기능 수행 후의 HTML 얻기

JS 렌더링까지 마친 뒤의 **현재 HTML** 반환

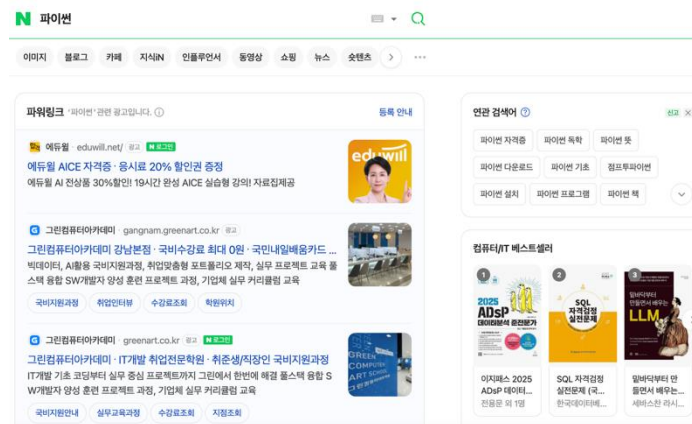
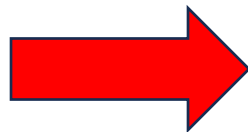
렌더링이 아직 완료되지 않았다면 일부 요소가 누락될 수 있으므로 대기(time.sleep() 등) 사용
로딩 타이밍이 중요

```
driver.page_source
```

스크롤, 입력, 버튼 클릭 등에 의해 동적으로 렌더링된 HTML을 가져오는 방법



동적 JS 렌더링



Selenium + BeautifulSoup 조합

Selenium만으로도 브라우저 조작 및 데이터 파싱이 가능하다! 하지만?

`find_element()`, `find_elements()`



Selenium



상대적으로 느리며, CPU/메모리 자원 사용이 크다
규모가 커짐에 따라 코드 복잡성 증가 및 비효율적

BeautifulSoup



HTML 파싱과 가공의 속도와 편의성을 제공함
대규모일수록 파싱 효율 증가 구조적인 클리닝 용이



Selenium으로 페이지를 열고, 사람처럼 브라우저 제어
BeautifulSoup으로 해당 페이지의 HTML을 빠르고 손쉽게 파싱

Selenium + BeautifulSoup 예시

```
# 1. 브라우저 실행
driver = webdriver.Chrome()
driver.get("https://quotes.toscrape.com/scroll")

# 2. 무한 스크롤 (Selenium 역할)
last_height = driver.execute_script("return document.body.scrollHeight")
while True:
    driver.execute_script("window.scrollTo(0, document.body.scrollHeight);")
    time.sleep(1.5)
    new_height = driver.execute_script("return document.body.scrollHeight")
    if new_height == last_height:
        break
    last_height = new_height

# 3. 로딩 완료 대기
WebDriverWait(driver, 5).until(
    EC.presence_of_all_elements_located((By.CLASS_NAME, "quote"))
)
```

무한 스크롤

```
# 4. 현재 페이지 HTML 가져오기 (Selenium → BeautifulSoup 전달)
html = driver.page_source
soup = BeautifulSoup(html, "html.parser")
```

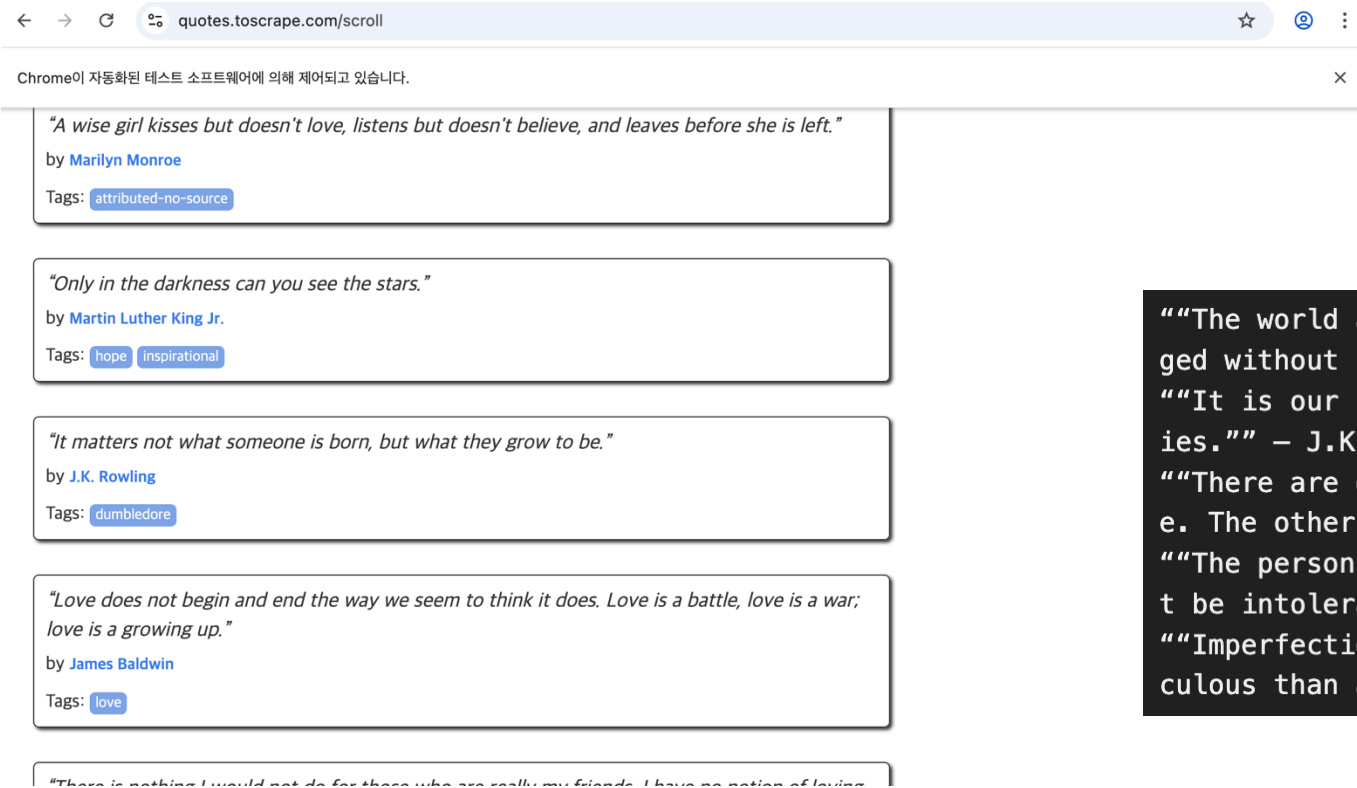
```
# 5. BeautifulSoup으로 데이터 추출
quotes = soup.select(".quote")

for q in quotes[:5]:
    text = q.select_one(".text").get_text(strip=True)
    author = q.select_one(".author").get_text(strip=True)
    print(f"{text} - {author}")

driver.quit()
```

데이터 파싱 (5개)

Selenium + BeautifulSoup 예시



무한 스크롤

““The world as we have created it is a process of our thinking. It cannot be changed without changing our thinking.”” – Albert Einstein
““It is our choices, Harry, that show what we truly are, far more than our abilities.”” – J.K. Rowling
““There are only two ways to live your life. One is as though nothing is a miracle. The other is as though everything is a miracle.”” – Albert Einstein
““The person, be it gentleman or lady, who has not pleasure in a good novel, must be intolerably stupid.”” – Jane Austen
““Imperfection is beauty, madness is genius and it's better to be absolutely ridiculous than absolutely boring.”” – Marilyn Monroe

파싱 결과(5개)



Selenium을 활용한 웹 크롤링

라이브러리 자체의 기능 복잡성보다는
어떤 식의 순서로 작동을 시킬지,
어떻게 반복할지,
어떤 조건으로 어떻게 움직일지
전략을 세우는 것이 중요



OpenAPI

OpenAPI란?

누구나 사용할 수 있도록 공개된 API를 말하며, 웹 사이트가 가진 기능을 모두가 이용할 수 있도록 공개한 프로그래밍 인터페이스

네이버 지도, 구글 맵이 대표적인 예이며,
코로나 때 생긴 라이브 코로나, 코로나 맵은 네이버 클라우드 플랫폼의 API 지원을 받고 있음

전체(92,771건)

파일데이터(69,489건)

오픈 API(11,535건)

표준데이터셋218개(9,961건)

연계데이터(1,786건)

수정일자순 ▾

10개씩 ▾

정렬

오픈 API (11,535건)

교통물류

공공기관

미리보기

XML

JSON

인천국제공항공사_화물기 운항 현황 상세 조회 서비스 Update

화물기에 대한 항공사,출/도착 공항명,출/도착 공항코드,코드웨어,변경일자,편명유니코드,편명,탑승구,마스터편명,현황,예정일자,터미널구분,항공기타입등의 정보를 D-DAY -3~+6일의 데이터 제공

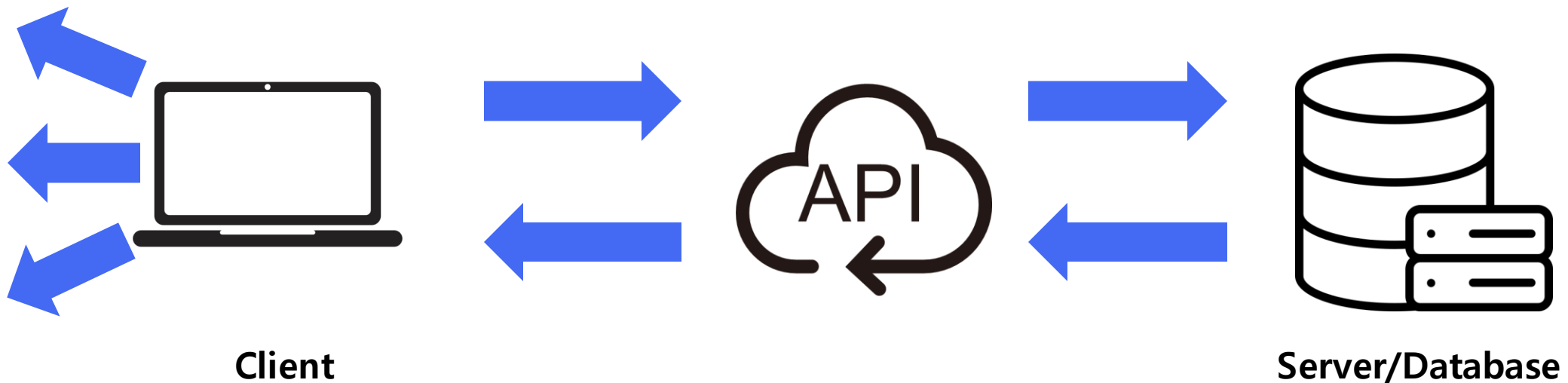
제공기관 인천국제공항공사 수정일 2024-09-02 조회수 878 활용신청 13 키워드 화물,인천,공항

활용신청

API(Application Programming Interface)

응용 프로그램끼리 서로 소통하기 위한 약속된 규칙

하나의 프로그램이 다른 프로그램의 기능이나 데이터를 표준화된 방식으로
요청할 수 있도록 하는 **통로**



API(Application Programming Interface)

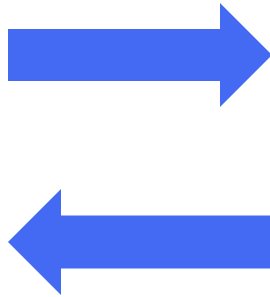
날씨 앱? 직접 기상 관측 X! 기상청 서버에서 요청한 데이터를 전달받고 활용



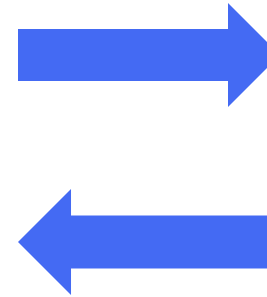
오늘 서울 날씨 알려줘!



Client



```
{  
  지역: 서울,  
  날씨: 흐림,  
  온도: 22.3  
  ...  
}
```



Server/Database

알겠어!

그러면 OpenAPI?

공개적으로 기능이나 데이터를 다른 프로그램에서 이용할 수 있게끔 하는 서비스



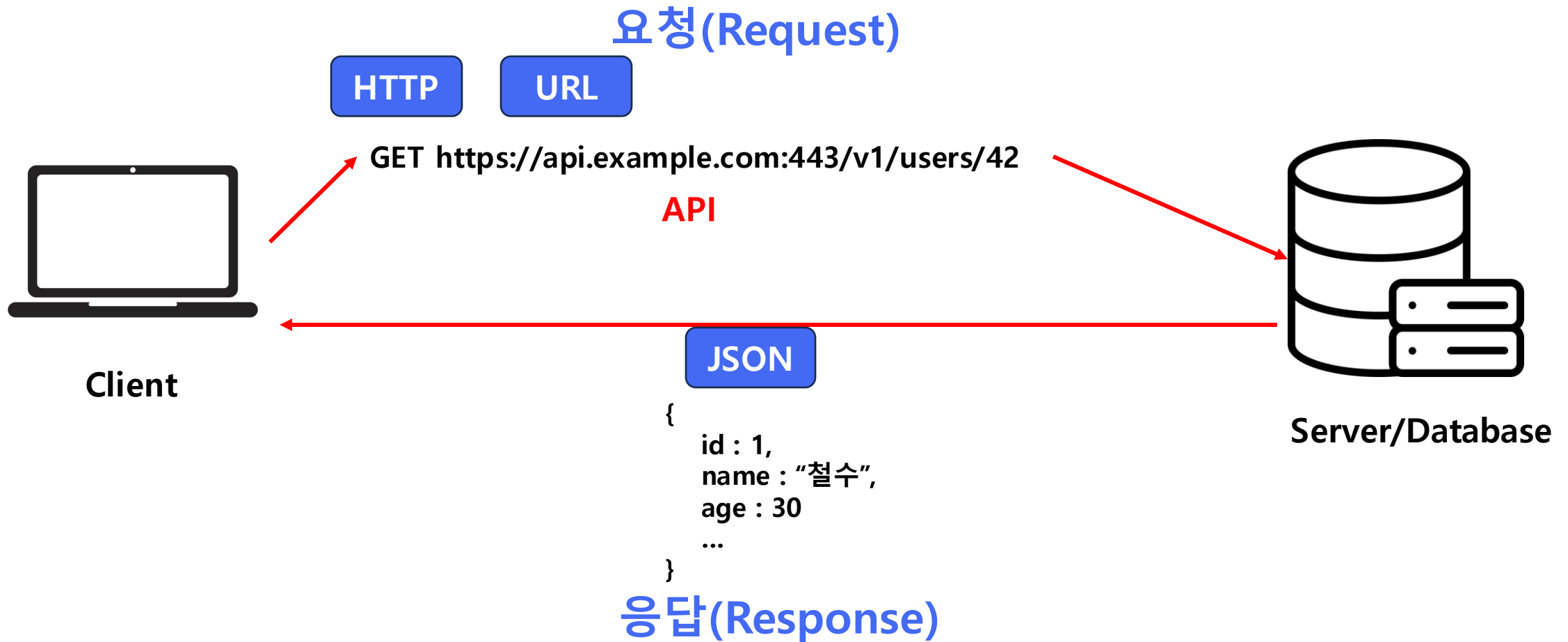
*대한민국 정부에서는 공공 데이터포털을 통해 도로명 주소 조회 서비스, 동네예보 정보 조회 서비스 등 굉장히 많은 데이터 제공



API의 구조

API의 기본적인 구조

웹 기반 클라이언트-서버 간의 요청/응답 구조



API의 구조 (1): End-point

클라이언트가 서버에 요청을 보낼 수 있는 정확한 주소

https://api.example.com:443/v1/users/42?active=true&limit=20

The diagram shows the URL **https://api.example.com:443/v1/users/42?active=true&limit=20** with blue brackets underneath. The first bracket is under 'https' and labeled '프로토콜'. The second bracket is under 'api.example.com' and labeled '서버의 도메인 주소 (host)'. The third bracket is under ':443' and labeled '포트'. The fourth bracket is under '/v1/users/42' and labeled 'path (접근 리소스)'. Above the second and fourth brackets, the Korean words '주소' and '경로' are written in red, with red lines pointing down to their respective brackets.

프로토콜 서버의 도메인 주소 (host) 포트 path (접근 리소스)

요청을 보낼 실제 주소
(서버 주소 + 자원 경로)

보통 HTTP, HTTPS 프로토콜을 많이 사용하며, 포트는 각 80, 443이며 생략 가능하다

API의 구조 (2): Query Parameter(= Query String)

요청 시, 웹 서버에 추가로 제공하는 파라미터(필터링, 옵션)

`https://api.example.com:443/v1/users/42?active=true&limit=20`
미름 값 미름 값
Query Parameters

Endpoint 주소 끝에 ?로 시작한다.

“파라미터이름=값” 형태로 전달하며,

여러 개의 파라미터를 넣는 경우에는 파라미터마다 &로 구분

API의 구조 (3): HTTP 메서드(Method)

서버에게 이 **자원**에 대해 **어떤 작업을 수행할 것인지** 알리는 **요청 방식**

GET *어떻게 요청을 보낼지* `https://api.example.com:443/v1/users/42?active=true&limit=20`

엔드포인트 및 쿼리 파라미터가 완성된 최종 URL을 갖고 요청

메서드	의미	특징	예시
GET	데이터 조회 (Read)	서버의 리소스만 요청	GET /users (사용자 목록 조회)
POST	데이터 생성 (Create)	서버에 새로운 리소스 추가	POST /users (새 사용자 등록)
PUT	데이터 전체 수정 (Update)	리소스 전체를 새 값으로 교체	PUT /users/1 (ID 1번 사용자 전체 수정)
PATCH	데이터 일부 수정 (Update)	리소스 일부 속성만 변경	PATCH /users/1 (ID 1번 사용자 일부 수정)
DELETE	데이터 삭제 (Delete)	특정 리소스 제거	DELETE /users/1 (ID 1번 사용자 삭제)

API의 구조 (3): HTTP 메서드(Method)

GET 요청과 POST 요청의 차이?

GET

- * URL에 직접적으로 요청 데이터(파라미터) 전달
(그대로 노출)

```
GET /v1/users?role=admin&page=2 HTTP/1.1  
Host: api.example.com
```

POST

- * HTTP Body(본문)에 숨겨서 데이터를 전달
(Body는 POST/PUT/PATCH에서 주로 사용)

```
POST /v1/users HTTP/1.1  
Host: api.example.com  
Content-Type: application/json
```

```
{  
  "name": "김철수",  
  "email": "kim@example.com"  
}
```

API의 구조 (4) : HTTP (요청/응답)헤더(Header)

데이터를 주고 받을 때, 본문(Body) 외의 **부가 정보(메타데이터)**를 전달하는 영역

- 요청 시 전달하는 **요청 헤더**
- 응답을 받을 때 전달되는 **응답 헤더**

헤더의 역할

- 요청 정보 전달
- 인증/보안 관리
- 콘텐츠 데이터 형식 지정
- 응답 정보 제공 *JSON, img ...*
- 캐싱 및 제어
- 추적 및 디버깅

```
GET /v1/users HTTP/1.1
```

```
Host: api.example.com
```

```
Authorization: Bearer sk_live_abcdefg → 인증키
```

```
Content-Type: application/json
```

```
User-Agent: python-requests/2.31.0
```

```
HTTP/1.1 200 OK
```

```
Content-Type: application/json
```

```
Cache-Control: no-cache
```

```
Date: Fri, 17 Oct 2025 15:00:00 GMT
```

API의 구조 (5): 응답 HTTP 상태 코드(Response HTTP Status Code)

요청을 보냈을 때, 서버가 그 요청을 어떻게 처리했는지를 코드로 표현

요청 결과 요청의 처리 결과를 3자리 숫자로 표현

첫번째 숫자로 대략적인 결과 범위 구분

- **1xx**: 정보의 의미로 요청을 계속 진행 중
100 ~ 199
- **2xx**: 성공의 의미로 **요청이 정상 처리됨**
- **3xx**: 리다이렉션의 의미로 다른 위치로 이동 필요
- **4xx**: **클라이언트 에러(Client Error)**로 **요청이 잘못됨**
- **5xx**: **서버 에러(Server Error)**로 **서버 내부에 문제가 발생**

```
HTTP/1.1 201 Created
Location: /v1/users/123
Content-Type: application/json
```

```
{
  "id": 123,
  "name": "김철수",
  "email": "kim@example.com"
}
```

```
HTTP/1.1 400 Bad Request
Content-Type: application/json
```

```
{
  "error": {
    "code": "invalid_parameter",
    "message": "필수 값 'email'이 누락되었습니다."
  }
}
```


API의 구조 (5): 응답 HTTP 상태 코드(Response HTTP Status Code)

대표적인 상태 코드

상태 코드	이름	의미
200 OK	요청 성공	일반적인 성공 응답
201 Created	생성 성공	POST로 리소스 새로 생성
400 Bad Request	잘못된 요청	클라이언트의 누락
401 Unauthorized	인증 실패	로그인/토큰 잘못됨
403 Forbidden	권한 없음	인증은 됐지만 권한 없음
404 Not Found	리소스 없음	존재하지 않는 URL/데이터 요청
500 Internal Server Error	서버 내부 오류	예상치 못한 오류
503 Service Unavailable	서비스 이용 불가	서버 과부화 또는 점검 중

API의 구조 (6): 응답 Body(Response Body)

요청에 따라 받은 **응답 결과물 데이터**

일반적인 웹 페이지에 GET 요청시, HTML 문서를 응답. *requests.get(url)*

하지만, 대부분의 API에 GET 요청 시 **데이터를 정해진 형식의 텍스트로 전달**

```
HTTP/1.1 201 Created
Location: /v1/users/123
Content-Type: application/json
```

```
{
  "id": 123,
  "name": "김철수",
  "email": "kim@example.com"
}
```

```
HTTP/1.1 400 Bad Request
Content-Type: application/json
```

```
{
  "error": {
    "code": "invalid_parameter",
    "message": "필수 값 'email'이 누락되었습니다."
  }
}
```

API의 응답 데이터 유형 (1): XML

태그 기반 마크업 데이터(eXtensible Markup Language)

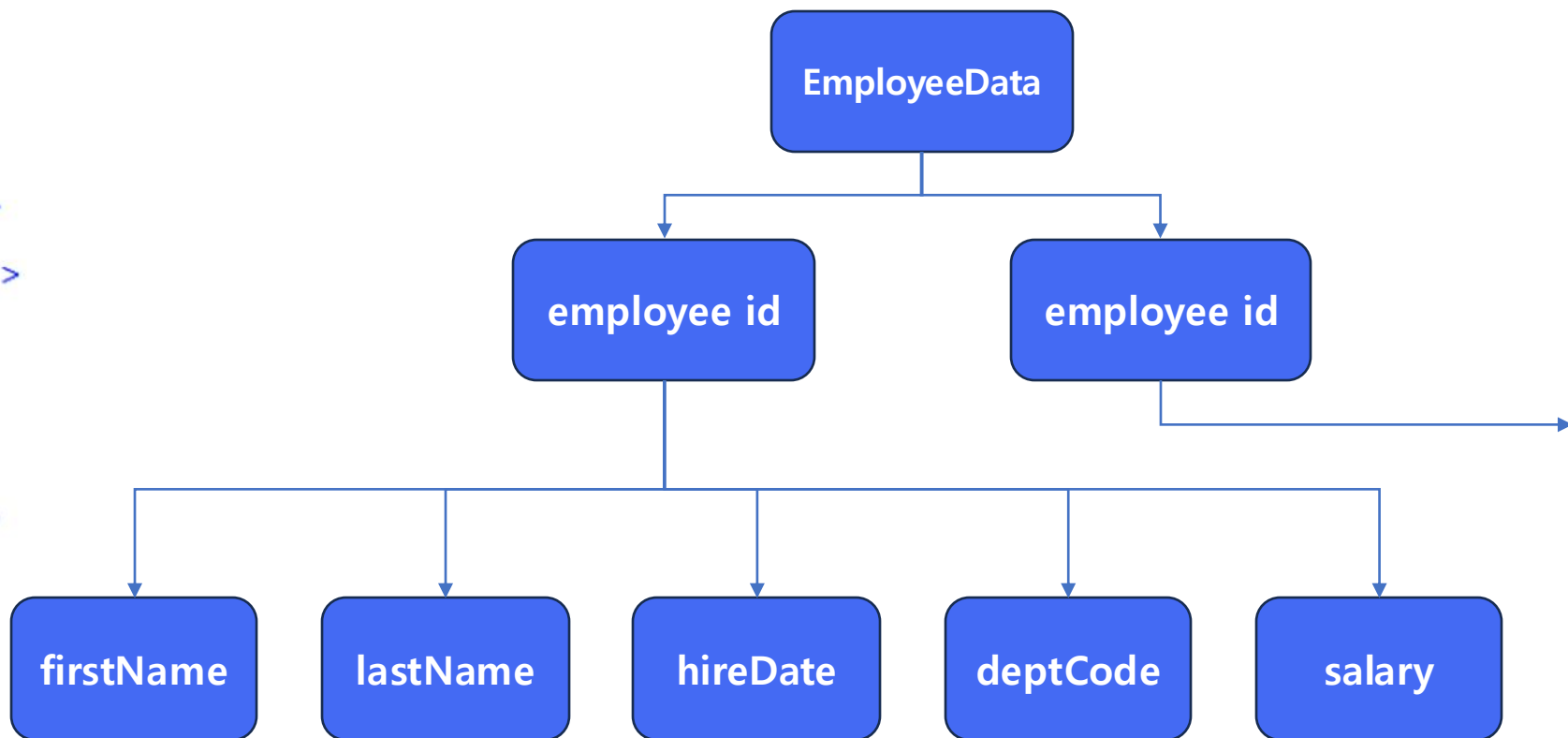
```
<?xml version="1.0" encoding="UTF-8"?>
- <EmployeeData>
  - <employee id="34594">
    <firstName>Heather</firstName>
    <lastName>Banks</lastName>
    <hireDate>1/19/1998</hireDate>
    <deptCode>BB001</deptCode>
    <salary>72000</salary>
  </employee>
  - <employee id="34593">
    <firstName>Tina</firstName>
    <lastName>Young</lastName>
    <hireDate>4/1/2010</hireDate>
    <deptCode>BB001</deptCode>
    <salary>65000</salary>
  </employee>
</EmployeeData>
```

- 태그로 감싸서 데이터 계층 표현
- 태그의 이름이 속성 이름
- 태그 쌍 안에 값 또는 또 다른 태그가 들어감

API의 응답 데이터 유형 (1): XML

트리 형태로 계층 표현

```
<?xml version="1.0" encoding="UTF-8"?>
- <EmployeeData>
  - <employee id="34594">
    <firstName>Heather</firstName>
    <lastName>Banks</lastName>
    <hireDate>1/19/1998</hireDate>
    <deptCode>BB001</deptCode>
    <salary>72000</salary>
  </employee>
  - <employee id="34593">
    <firstName>Tina</firstName>
    <lastName>Young</lastName>
    <hireDate>4/1/2010</hireDate>
    <deptCode>BB001</deptCode>
    <salary>65000</salary>
  </employee>
</EmployeeData>
```



API의 응답 데이터 유형 (2): JSON

데이터를 **Key-Value 쌍으로 표현**하는 텍스트 기반 형식

```
{
  "EmployeeData": {
    "employee": [
      {
        "id": "34594",
        "firstName": "Heather",
        "lastName": "Banks",
        "hireDate": "1/19/1998",
        "deptCode": "BB001",
        "salary": 72000
      },
      {
        "id": "34593",
        "firstName": "Tina",
        "lastName": "Young",
        "hireDate": "4/1/2010",
        "deptCode": "BB001",
        "salary": 65000
      }
    ]
  }
}
```

- **키:값** 형태로 속성 하나를 정의
- 가장 일반화된 구조 응답 형식(API 표준 응답 포맷)
- 가볍고 읽기 쉬움
- 계층적으로 데이터 표현 가능

API의 응답 데이터 유형 (3): CSV

콤마로 값을 구분

```
id,firstName,lastName,hireDate,deptCode,salary  
34594,Heather,Banks,1998-01-19,BB001,72000  
34593,Tina,Young,2010-04-01,BB001,65000
```



OpenAPI의 활용

OpenAPI의 활용

대부분의 OpenAPI 서비스를 사용하기 위해서는 **API Key가 필요하다**



API Key?

OpenAPI 서비스에 접근하기 위한 **사용자 고유 인증 키**

사용자를 **식별**하고 **권한을 부여**하기 위함

OpenAPI의 활용: API Key

API Key의 역할

인증
(Authentication)

요청을 보내는 사용자가 누구인지를 식별

권한
(Authorization)

사용자가 접근 가능한 데이터/엔드포인트 제한

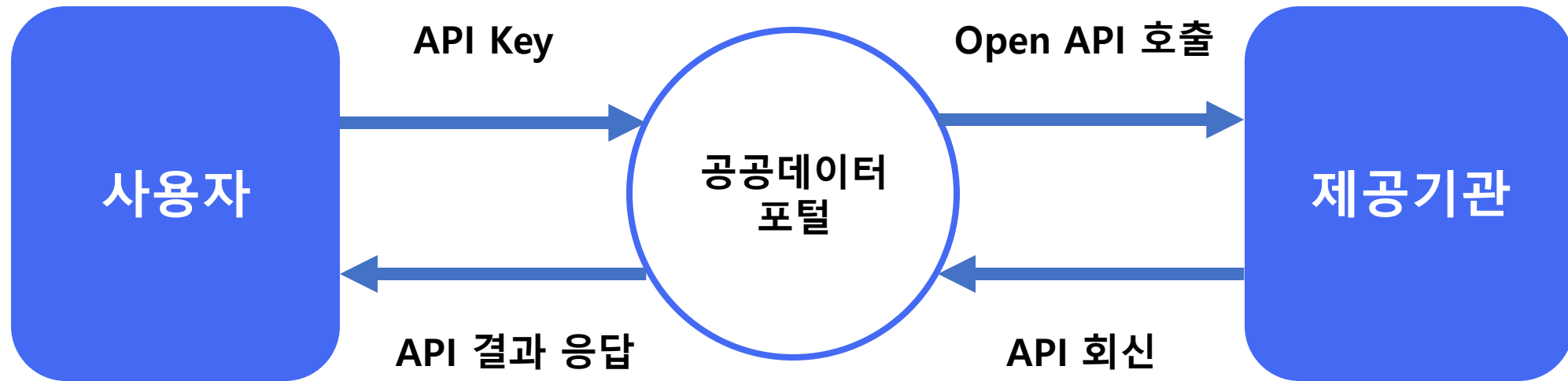
사용량 관리
(Quota)

하루 요청 횟수 제한 (ex: 10,000건/일)

추적 및 통계

API 제공자가 사용자별 호출 로그 관리

공공데이터 포털의 OpenAPI



서비스에 따라 API Key는 Query Parameter 또는 Header를 통해 전달

공공데이터 포털의 OpenAPI

원하는 API에 대해 활용 신청을 통해 API Key를 발급

전체(5건)	파일데이터(1건)	오픈 API(4건)	표준데이터셋0개(0건)	연계데이터(0건)
--------	-----------	------------	--------------	-----------

보건의료

국가행정기관

국가중점

미리보기

XML

JSON

보건복지부_코로나19 확진자 성별 연령별 현황

※ 데이터 출처: 질병관리청(코로나바이러스감염증-19 홈페이지) ※ 중앙재난안전대책본부(질병관리청)의 코로나19 위기단계 하향 및 방역 조치 전환 결정에 따라 '23년 6월 1일부터는

제공기관

보건복지부

수정일

2023-12-26

조회수

20640

활용신청

919

키워드

보건의료,코로나19,확진자

활용신청

활용목적 선택

*표시는 필수 입력항목입니다.

*활용목적

☒ 웹 사이트 개발 ☐ 앱개발 (모바일,솔루션등) ☐ 기타 ☐ 참고자료 ☐ 연구(논문 등)

코로나 데이터를 분석하고자 합니다

상세기능정보 선택

☒	상세기능	설명	일일 트래픽
☒	보건복지부_코로나19 확진자 성별 연령별 현황	코로나19 감염현황을 성별 및 연령별로 조회하는 기능	10000

라이선스 표시

*이용허락범위

☒ 동의합니다.

공공데이터 포털의 OpenAPI

발급받은 Key를 바탕으로 해당 API 규격/구조에 맞춰 요청

- 활용승인 절차 개발단계 : 허용 / 운영단계 : 허용
- 신청가능 트래픽 10000000 / 운영계정은 활용사례 등록시 신청하면 트래픽 증가 가능
- 요청주소
- 서비스URL <http://openapi.data.go.kr/openapi/service/rest/Covid19/getCovid19GenAgeCaseInfJson>

요청 대상 End-point

요청변수(Request Parameter)

항목명(국문)	항목명(영문)	항목크기	항목구분	샘플데이터
서비스키	ServiceKey	4	필수	-
페이지 번호	pageNo	4	옵션	1
한 페이지 결과 수	numOfRows	4	옵션	10
데이터 생성일 시작범위	startCreateDt	30	옵션	20200310
데이터 생성일 종료범위	endCreateDt	30	옵션	20200414

발급받은 서비스키(API Key)와 함께 요청

*필수 항목을 반드시 추가

*옵션 항목은 선택적으로 추가

serviceKey=3A7F4F8A-51BC-4D92-BE0A-FC7A24C73FA4

Key는 긴 문자열 형태(UUID)

API 문서 Example: 네이버 지도 API

API 요청을 어떻게 보내야 할지 규격 가이드(Spec)를 작성한 문서

API URL

요청 API URL은 다음과 같습니다.

End point url 명시

HTTP

COPY

Static Map: <https://maps.apigw.ntruss.com/map-static/v2>
Directions 5: <https://maps.apigw.ntruss.com/map-direction/v1>
Directions 15: <https://maps.apigw.ntruss.com/map-direction-15/v1>
Geocoding: <https://maps.apigw.ntruss.com/map-geocode/v2>
Reverse Geocoding: <https://maps.apigw.ntruss.com/map-reversegeocode/v2>

요청 헤더

헤더에 대한 설명은 다음과 같습니다.

필드	필수 여부	설명
x-ncp-apigw-api-key-id	Required	네이버 클라우드 플랫폼 콘솔에서 Application 등록 후 발급받은 Client ID
x-ncp-apigw-api-key	Required	네이버 클라우드 플랫폼 콘솔에서 Application 등록 후 발급받은 Client Secret

API 문서 Example: 네이버 지도 API

API 요청/응답 규격 가이드를 작성한 문서

요청

요청 형식을 설명합니다. 요청 형식은 다음과 같습니다.

메서드	URI
GET	/geocode

요청 헤더

헤더에 대한 설명은 다음과 같습니다.

필드	필수 여부	설명
x-ncp-apigw-api-key-id	Required	네이버 클라우드 플랫폼 콘솔에서 Application 등록 후 발급받은 Client ID
x-ncp-apigw-api-key	Required	네이버 클라우드 플랫폼 콘솔에서 Application 등록 후 발급받은 Client Secret

API 문서 Example: 네이버 지도 API

요청 쿼리 파라미터

요청 쿼리 파라미터에 대한 설명은 다음과 같습니다.

필드	타입	필수 여부	설명
query	String	Required	검색할 주소
coordinate	String	Optional	검색 중심 좌표(경도,위도) 입력한 좌표와 근접한 순으로 검색 결과 표시
filter	Integer	Optional	검색 결과 필터 - HCODE BCODE - hCODE: 행정동 코드 - BCODE: 법정동 코드 - 필터 타입@코드1;코드2;..... 형식으로 입력 <예시> HCODE@4113554500;4113555000
language	String	Optional	응답 결과 언어 - kor (기본값) eng - kor: 한국어 - eng: 영어
page	Number	Optional	페이지 번호 1 (기본값)

API 문서 Example: 네이버 지도 API

응답 바디

응답 바디에 대한 설명은 다음과 같습니다.

필드	타입	필수 여부	설명
status	String	-	응답 코드
meta	Object	-	메타 데이터
meta.totalCount	Number	-	응답 결과 개수
meta.page	Number	-	현재 페이지 번호
meta.count	Number	-	페이지 내 결과 개수
addresses	Array	-	주소 정보 목록
errorMessage	String	-	오류 메시지 • 500 오류(알 수 없는 오류) 발생 시에만 표시



응답 상태 코드

응답 상태 코드에 대한 설명은 다음과 같습니다.

HTTP 상태 코드	코드	메시지	설명
200	OK	-	요청 처리 성공. 정상 응답
400	INVALID_REQUEST	-	요청 오류
500	SYSTEM_ERROR	Unexpected Error	알 수 없는 오류

API 문서 Example: 네이버 지도 API

요청 예시

요청 예시는 다음과 같습니다.

Shell

COPY

```
curl --location --request GET 'https://maps.apigw.ntruss.com/map-geocode/v2/geocode?query=분당구 불정로 6' \  
--header 'x-ncp-apigw-api-key-id: {API Key ID}' \  
--header 'x-ncp-apigw-api-key: {API Key}' \  
--header 'Accept: application/json'
```

```
{  
  "status": "OK",  
  "meta": {  
    "totalCount": 1,  
    "page": 1,  
    "count": 1  
  },  
  "addresses": [  
    {  
      "roadAddress": "경기도 성남시 분당구 불정로 6 NAVER그린팩토리",  
      "jibunAddress": "경기도 성남시 분당구 정자동 178-1 NAVER그린팩토리",  
      "englishAddress": "6, Buljeong-ro, Bundang-gu, Seongnam-si, Gyeonggi-do, Republic of Korea",  
      "addressElements": [  
        {  
          "types": [  
            "SIDO"  
          ],  
          "longName": "경기도",  
          "shortName": "경기도",  
          "code": ""  
        },  
        {  
          "types": [  
            "SIGUGUN"  
          ],  
          "longName": "성남시 분당구",  
          "shortName": "성남시 분당구",  
          "code": ""  
        },  
        {  
          "types": [  
            "DONGMYUN"  
          ],  
          "longName": "분당구 불정로 6 NAVER그린팩토리",  
          "shortName": "분당구 불정로 6 NAVER그린팩토리",  
          "code": ""  
        }  
      ]  
    }  
  ]  
}
```

requests 라이브러리 활용하여 요청/응답

```
import requests

url = "https://api.openweathermap.org/data/2.5/weather"

params = {
    "q": "Seoul",
    "appid": API_KEY,
    "units": "metric"
}

res = requests.get(url, params=params)

if res.status_code == 200:
    data = res.json()
    print("도시명:", data["name"])
    print("기온:", data["main"]["temp"], "°C")
else:
    print('Error:', res.text)
```

End-point 정의(API URL)

Query Parameter 정의

GET 요청 후, 응답 리턴

응답 상태 코드가 200이면,
* 응답 바디가 JSON 포맷이면
딕셔너리 형태로 반환

그렇지 않은 경우(에러) 다른 처리

requests 라이브러리 활용하여 요청/응답

```
import requests
import json

# 1. 엔드포인트 (요청 URL)
url = "https://api.openweathermap.org/data/2.5/weather"

# 2. 쿼리 파라미터 (params)
params = {
    "q": "Seoul",          # 도시 이름
    "appid": "YOUR_API_KEY", # API Key
    "units": "metric"      # 섭씨 온도 단위
}

# 3. 요청 헤더 (headers)
headers = {
    "Accept": "application/json",
    "User-Agent": "PythonRequestsDemo/1.0"
}

# 4. GET 요청 보내기
response = requests.get(url, headers=headers, params=params)

# 5. 응답 정보 출력
print("응답 코드(Status Code):", response.status_code)
print("응답 헤더(Response Headers):")
for key, value in response.headers.items():
    print(f"  {key}: {value}")

# 6. 응답 바디 확인
print("\n응답 바디(Response Body):")
try:
    data = response.json() # JSON 형태일 경우
    print(json.dumps(data, indent=2, ensure_ascii=False))
except ValueError:
    print(response.text)   # JSON이 아닐 경우 텍스트로 출력
```

응답 코드(Status Code): 200

응답 헤더(Response Headers):

Content-Type: application/json; charset=utf-8

Server: openresty

Date: Fri, 18 Oct 2025 09:00:00 GMT

응답 바디(Response Body):

```
{
  "coord": {"lon": 126.9778, "lat": 37.5683},
  "weather": [{"main": "Clouds", "description": "구름 조금"}],
  "main": {"temp": 21.3, "humidity": 68},
  "name": "Seoul"
}
```

Github, 블로그 등 공개 저장소에 Key 포함 또는 공유 금지

감사합니다!